

BERT Vector Generation and Visualization Toolkit

Data Engineering Team

June 18, 2026

Overview

This document contains a comprehensive toolkit of scripts utilized for generating vector embeddings from the Kurdish medical dataset using the BERT model, and visualizing the semantic relationships between these embeddings. Below is a summary table containing links to the code for each step in the pipeline, alongside an explanation of its purpose.

Script Name	Section Link	Purpose / Why We Use It
Vector Pipeline		
Vector Generator	Section 1	Generates vector embeddings for a batch of texts using the BERT large model.
Vector Visualization	Section 2	Computes cosine similarity between vectors and builds an interactive pyvis network graph.
Setup & Environment		
Setup Script	Section 3	Prepares the environment by installing dependencies and caching the BERT model.
Model Loader	Section 4	A dedicated script to trigger the Hugging Face download of the BERT model for caching.
Requirements	Section 5	Lists the necessary Python packages for the pipeline.
Environment Variables	Section 6	Configuration file for specifying model and paths.

Table 1: Vector generation and setup scripts categorized by their functional domain.

1 Vector Generator

Purpose: We use this script to generate vector embeddings for the Kurdish medical dataset using the `bert-large-cased` model. It includes batching, checkpointing, and GPU acceleration.

Listing 1: `vector_generator.py`

```
1 import json
2 import os
3 import logging
4 from typing import List, Dict
5 from tqdm import tqdm
6 import torch
7 from transformers import AutoTokenizer, AutoModel
8
9 # Setup Logging
10 logging.basicConfig(
11     level=logging.INFO,
12     format='%(asctime)s - %(levelname)s - %(message)s',
13     handlers=[
14         logging.FileHandler("vector_pipeline.log"),
15         logging.StreamHandler()
16     ]
17 )
18
19 # Configuration
20 # We use bert-large-cased as requested for BERT
21 MODEL_NAME = "bert-large-cased"
22 DATASET_PATH = "../just_data_vector/kurdish_medical_corpus_kmc.json"
23 OUTPUT_PATH = "kurdish_medical_vectors.jsonl"
24 CHECKPOINT_PATH = "processed_ids.txt"
25
26 class VectorGenerator:
27     def __init__(self):
28         logging.info(f"Downloading/Loading Model: {MODEL_NAME}")
29         # Initialize tokenizer and model
30         self.tokenizer = AutoTokenizer.from_pretrained(MODEL_NAME)
31         self.model = AutoModel.from_pretrained(MODEL_NAME)
32
33         # Check if GPU is available
34         self.device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
35         self.model.to(self.device)
36         self.model.eval()
37         logging.info(f"Model loaded and moved to {self.device}")
38
39         self.processed_ids = self._load_checkpoints()
40
41     def _load_checkpoints(self) -> set:
42         if os.path.exists(CHECKPOINT_PATH):
43             with open(CHECKPOINT_PATH, 'r') as f:
44                 return set(line.strip() for line in f)
45         return set()
46
47     def _save_checkpoint(self, entry_id: str):
48         with open(CHECKPOINT_PATH, 'a') as f:
49             f.write(f"{entry_id}\n")
50         self.processed_ids.add(entry_id)
51
```

```

52 def clean_text(self, text: str) -> str:
53     if not text:
54         return ""
55     # Basic cleanup
56     text = text.replace('\n', ' ')
57     text = ' '.join(text.split())
58     return text
59
60 def generate_vectors_batch(self, texts: List[str]) -> List[List[float]]:
61     """
62     Generates vector embeddings for a batch of texts using XLM-R.
63     """
64     if not texts:
65         return []
66
67     try:
68         # Tokenize input text in batches
69         inputs = self.tokenizer(
70             texts,
71             return_tensors="pt",
72             truncation=True,
73             max_length=512,
74             padding=True
75         ).to(self.device)
76
77         # Generate embeddings
78         with torch.no_grad():
79             outputs = self.model(**inputs)
80
81         # Mean pooling of the last hidden states
82         attention_mask = inputs['attention_mask'].unsqueeze(-1).expand(outputs.
last_hidden_state.size()).float()
83         sum_embeddings = torch.sum(outputs.last_hidden_state * attention_mask, 1)
84         sum_mask = torch.clamp(attention_mask.sum(1), min=1e-9)
85         mean_pooled = sum_embeddings / sum_mask
86
87         # Convert to list of lists of floats
88         vectors = mean_pooled.cpu().numpy().tolist()
89         return vectors
90
91     except Exception as e:
92         logging.warning(f"Vector generation failed for batch... Error: {e}")
93         return [[] for _ in texts]
94
95 def store_vectors_batch(self, entry_ids: List[str], vectors: List[List[float]]):
96     """
97     Appends a batch of generated vectors to the JSONL file.
98     """
99     with open(OUTPUT_PATH, 'a', encoding='utf-8') as f:
100         for entry_id, vector in zip(entry_ids, vectors):
101             if not vector:
102                 continue
103             record = {
104                 "id": entry_id,
105                 "vector": vector
106             }
107             f.write(json.dumps(record, ensure_ascii=False) + '\n')
108
109 def process_dataset(self, batch_size: int = 32):

```

```

110         logging.info(f"Loading dataset from {DATASET_PATH}...")
111     try:
112         with open(DATASET_PATH, 'r', encoding='utf-8') as f:
113             data = json.load(f)
114     except Exception as e:
115         logging.error(f"Could not load JSON: {e}")
116         return
117
118     logging.info(f"Processing {len(data)} entries with batch_size={batch_size}.
Found {len(self.processed_ids)} already processed.")
119
120     batch_ids = []
121     batch_texts = []
122
123     for entry in tqdm(data):
124         entry_id = entry.get('id')
125         if not entry_id or entry_id in self.processed_ids:
126             continue
127
128         raw_text = entry.get('response', '')
129         if not raw_text:
130             self._save_checkpoint(entry_id)
131             continue
132
133         cleaned_text = self.clean_text(raw_text)
134
135         batch_ids.append(entry_id)
136         batch_texts.append(cleaned_text)
137
138         if len(batch_ids) >= batch_size:
139             vectors = self.generate_vectors_batch(batch_texts)
140             self.store_vectors_batch(batch_ids, vectors)
141
142             # Save checkpoints
143             for b_id in batch_ids:
144                 self._save_checkpoint(b_id)
145
146             batch_ids = []
147             batch_texts = []
148
149         # Process remaining
150         if batch_ids:
151             vectors = self.generate_vectors_batch(batch_texts)
152             self.store_vectors_batch(batch_ids, vectors)
153             for b_id in batch_ids:
154                 self._save_checkpoint(b_id)
155
156     logging.info("Pipeline finished successfully.")
157
158 if __name__ == "__main__":
159     generator = VectorGenerator()
160     try:
161         generator.process_dataset()
162     except KeyboardInterrupt:
163         logging.info("Process interrupted by user. Progress saved.")
164     except Exception as e:
165         logging.error(f"Fatal error: {e}")

```

2 Vector Visualization

Purpose: We use this script to compute the cosine similarity between the generated vectors and visualize their relationships interactively using pyvis. It filters relationships based on a similarity threshold.

Listing 2: visualize_vector_relationships.py

```
1 import json
2 import numpy as np
3 from sklearn.metrics.pairwise import cosine_similarity
4 from pyvis.network import Network
5 import logging
6
7 logging.basicConfig(level=logging.INFO, format='%(asctime)s - %(message)s')
8
9 VECTOR_FILE = "kurdish_medical_vectors.jsonl"
10 OUTPUT_HTML = "vector_relationships.html"
11 SIMILARITY_THRESHOLD = 0.85 # Adjust this to show more/fewer relationships
12
13 def load_vectors():
14     ids = []
15     vectors = []
16     logging.info(f"Loading vectors from {VECTOR_FILE}...")
17     try:
18         with open(VECTOR_FILE, 'r', encoding='utf-8') as f:
19             for line in f:
20                 data = json.loads(line)
21                 ids.append(str(data['id']))
22                 vectors.append(data['vector'])
23     except FileNotFoundError:
24         logging.error(f"{VECTOR_FILE} not found. Please run vector_generator.py first.")
25     return [], []
26     return ids, np.array(vectors)
27
28 def build_graph():
29     ids, vectors = load_vectors()
30     if not ids:
31         return
32
33     logging.info(f"Loaded {len(ids)} vectors. Computing similarities...")
34
35     # Compute cosine similarity between all vectors
36     similarity_matrix = cosine_similarity(vectors)
37
38     # Initialize pyvis network
39     logging.info("Building interactive graph...")
40     net = Network(height="100vh", width="100%", bgcolor="#222222", font_color="white")
41
42     # Add nodes
43     for i, node_id in enumerate(ids):
44         # We only add nodes if they have at least one connection, to keep it clean.
45         # But for now, we'll add all of them, or just the ones that connect.
46         pass
47
48     # Find relationships (edges) based on similarity using optimized NumPy operations
49     edges = []
```

```

50     connected_nodes = set()
51
52     # Get the upper triangle of the similarity matrix (excluding diagonal)
53     row_indices, col_indices = np.triu_indices_from(similarity_matrix, k=1)
54
55     # Filter indices where similarity is above threshold
56     valid_mask = similarity_matrix[row_indices, col_indices] >= SIMILARITY_THRESHOLD
57     valid_rows = row_indices[valid_mask]
58     valid_cols = col_indices[valid_mask]
59     valid_sims = similarity_matrix[valid_rows, valid_cols]
60
61     # Optional safety measure: limit max edges so the browser doesn't crash on massive
62     # graphs
63     MAX_EDGES = 10000
64     if len(valid_sims) > MAX_EDGES:
65         logging.warning(f"Found {len(valid_sims)} edges, which might freeze the
66 browser. Limiting to top {MAX_EDGES} strongest edges.")
67         # Get indices of top MAX_EDGES similarities
68         top_indices = np.argsort(valid_sims)[-MAX_EDGES:]
69         valid_rows = valid_rows[top_indices]
70         valid_cols = valid_cols[top_indices]
71         valid_sims = valid_sims[top_indices]
72
73     for r, c, sim in zip(valid_rows, valid_cols, valid_sims):
74         edges.append((int(r), int(c), float(sim)))
75         connected_nodes.add(int(r))
76         connected_nodes.add(int(c))
77
78     logging.info(f"Found {len(edges)} strong relationships (similarity >= {
79 SIMILARITY_THRESHOLD}).")
80
81     # Add only connected nodes to keep the graph readable
82     for node_idx in connected_nodes:
83         net.add_node(node_idx, label=f"Document {ids[node_idx]}", title=f"ID: {ids[
84 node_idx]}")
85
86     # Add edges
87     for i, j, sim in edges:
88         # Weight the edge based on similarity
89         weight = (sim - SIMILARITY_THRESHOLD) / (1.0 - SIMILARITY_THRESHOLD) * 5
90         net.add_edge(i, j, value=weight, title=f"Similarity: {sim:.2f}")
91
92     # Configure physics for a nice Neo4j-like layout
93     net.barnes_hut(gravity=-8000, central_gravity=0.3, spring_length=250)
94
95     # Save the HTML file
96     net.save_graph(OUTPUT_HTML)
97     logging.info(f"Graph saved to {OUTPUT_HTML}!")
98     logging.info(f"Open {OUTPUT_HTML} in any web browser to view the relationships.")
99
100 if __name__ == "__main__":
101     build_graph()

```

3 Setup Script

Purpose: We use this shell script to prepare the environment, install required Python dependencies from `requirements.txt`, create a `.env` template, and cache the BERT model.

Listing 3: setup.sh

```
1  #!/bin/bash
2
3  # KMC Vector Generation Setup Script
4
5  echo "👉 Starting setup for KMC Vector Pipeline..."
6
7  # 1. Check for Python
8  if ! command -v python3 &> /dev/null; then
9      echo "👉 Python3 is not installed. Please install it first."
10     exit 1
11 fi
12
13 # 2. Install Requirements
14 echo "👉 Installing dependencies from requirements.txt..."
15 pip install --upgrade pip
16 pip install -r requirements.txt
17
18 # 3. Create .env template if it doesn't exist
19 if [ ! -f .env ]; then
20     echo "👉 Creating .env template..."
21     echo "MODEL_NAME=bert-large-cased" > .env
22     echo "DATASET_PATH=../just_data_vector/kurdish_medical_corpus_kmc.json" >> .env
23     echo "OUTPUT_PATH=kurdish_medical_vectors.jsonl" >> .env
24 fi
25
26 # 4. Download and Cache BERT Model
27 echo "👉 Downloading and caching the 'bert-large-cased' model..."
28 python3 -c "
29 from transformers import AutoTokenizer, AutoModel
30 print('Downloading tokenizer...')
31 AutoTokenizer.from_pretrained('bert-large-cased')
32 print('Downloading model...')
33 AutoModel.from_pretrained('bert-large-cased')
34 print('Model cached successfully!')
35 "
36
37 echo "👉 Setup complete!"
38 echo "👉 You can now run the pipeline: 'python vector_generator.py'"
```

4 Model Loader

Purpose: A dedicated script used to download and cache the `bert-large-cased` model from Hugging Face into the local environment before running the main pipeline.

Listing 4: load_model_gpu_1.sh

```

1  #!/bin/bash
2
3  # Load Model into Cache
4
5  echo -e "\033[1;33m[*] Action: Caching 'bert-large-cased' model...\033[0m"
6
7  # We use python to trigger the Hugging Face download
8  python3 -c "
9  from transformers import AutoTokenizer, AutoModel
10 print('Downloading tokenizer...')
11 AutoTokenizer.from_pretrained('bert-large-cased')
12 print('Downloading model...')
13 AutoModel.from_pretrained('bert-large-cased')
14 print('Done!')
15 "
16
17 echo -e "\033[1;32m[SUCCESS] Model 'bert-large-cased' is cached successfully!\033[0m"

```

5 Requirements

Purpose: Defines the required Python packages for running the vector generation and visualization pipeline.

Listing 5: requirements.txt

```

1  transformers
2  torch
3  tqdm
4  scikit-learn
5  pyvis

```

6 Environment Variables

Purpose: The configuration settings used by the vector generator.

Listing 6: .env

```

1  MODEL_NAME=bert-large-cased
2  DATASET_PATH=../just_data_vector/kurdish_medical_corpus_kmc.json
3  OUTPUT_PATH=kurdish_medical_vectors.jsonl

```